Lecture 10 - Cryptography (Private key crypto, one-way functions, PRGs)

ok, so...

cryptography is a fundamental, essential aspect of our shared political reality, since at least ancient rome

The basic issue is this: you want to send messages through a public channel (e.g. radio waves) in such a way that no-one but the person you want to send the message to can know what the message is

Famously, the nazis used a cryptographic aparatus called the "enigma". Also famously, Alan Turing's team, working for the British (MI5? Something) secret service, cracked the enigma code, with the help of a purpose-built computing aparatus, designed by Turing, called the "bombe".

It is quite possible that without this the germans would have won WWII.

After the victory of the allied forces, it was quite clear to every military mind in the US that cryptography was an essential part of a capable military. So they hired several mathematicians to work on cryptography. One such mathematician, John Nash, made the following conjecture in 195..4?

Nash's Conjecture. There exists algorithm to [break this set of cyphers that Nash constructed] that runs in better than exponential time.

As we will (roughly) see, his conjecture, if true, would imply that P != NP.

Basic Scenario

We have three parties: Alice, Bob, and Eve(sdropper).

Alice wants to send a message $x \in \{0,1\}^m$ to Bob. For this purpose, Alice and Bob have at their disposal a communication channel. We abstract away all the difficulties of communication having to do with error correction (for this see information theory, source coding theorem, etc), and assume that the channel works perfectly: when Alice puts one bit on her side of the channel, Bob receives that same bit on his side.

The issue is: the channel is public. So we assume there is a malicious party, called Eve, who can see everything that is sent through the channel. Alice wants to use the channel in such a way that allows Bob to learn x, but not Eve. What can they do?

Nothing, without some assumption.

We will assume, then, that Alice and Bob share a secret, known to them but unknown to Eve.

This secret is called a "key". We will assume that the key is a string $k \in \{0,1\}^n$.

We then come to the following definition:

Definition (Encryption Scheme). An encryption scheme is a pair of functions $E : \{0,1\}^n \times \{0,1\}^m \to \{0,1\}^*$ (the encoder) and $D : \{0,1\}^n \times \{0,1\}^* \to \{0,1\}^m$ such that, for all $x \in \{0,1\}^m$, $k \in \{0,1\}^n$, $D(k, E(k,x)) = x$. (i.e., knowing the key one encode and decode.)

How does one guarantee that such a scheme is secure? The first person to come up with a definition was Claude Shannon.

Definition (perfect security). An encryption scheme (E, D) is *perfectly secure* if for every pair of messages $x, x' \in \{0,1\}^m$ the two distributions $E(U_n, x)$ and $E(U_n, x')$ are identical.

(here, $E(U_n, x)$ we mean the distribution obtained by choosing the n-bit key $k \in \{0,1\}^n$ uniformly at random and outputting $E(k, x)$)

makes sense?

You see, from the point of view of Eve, the key is unknown, which we interpret as saying that every key $k \in \{0,1\}^m$ is equally likely. Then Eve expects to see $E(U_n, x)$ on the public communication channel. But if the two distributions $E(U_n, x)$ and $E(U_n, x')$ are the exact same distribution, then Eve, not knowing the key, cannot distinguish which of the two scenarios she is in (Alice sent x vs Alice sent x').

Example of a perfectly secure cryptosystem: the one-time pad

The *one-time pad* is the following cryptosystem. The key length n is equal to the message length m, and we define $E(k, x) = k \oplus x$ (the bitwise XOR) and $D(k, y) = k \oplus y$.

Then, for every x $E(U_m, x) = U_m$ is the uniform distribution, and hence the cryptosystem is perfectly secure.

Great, right? What's the problem?

The problem is not that someone might have access to the key. That is an *unavoidable problem*. Every cryptosystem can be broken if the evesdropper knows the key.

If I want to encrypt 1Gb of data, I need a shared key which is 1Gb long!

That means that for Alice and Bob to communicate securely, they need to decide beforehand how much they are going to communicate in total.

Furthermore, if you want to communicate with 1M people, you need 1M keys.

Etc.

This problem is insurmountable: for homework you are asked to prove that if (E, D) is a cryptosystem with $n < m$, then it is not perfectly secure. (This actually follows as a corollary of what we will see below.)

Computational Complexity enters the picture

**Lemma (secure encryption is impossible if $\mathsf{P} = \mathsf{NP}$). Let (E, D) be any encryption scheme where E and D are both computable in polynomial time. Suppose that $\mathsf{P} = \mathsf{NP}$. Then there exists a polynomial-time algorithm A such that for every input length m there exist two messages $x_0, x_1 \in \{0,1\}^m$ with

$$\Pr_{b,k}[A(E(k, x_b)) = b] \geq \frac{3}{4}$$

I.e., by looking at the encoded message $E(k, x_b)$, A can guess which message was sent (if the security was perfect the above probability would be $1/2$ at most).

Proof.

Let $x_0 = 0^m$, and let $S = E(\{0,1\}^n, 0^m)$ be the set of possible encodings of the message x_0.

Our algorithm is simple: $A(y) = 0$ iff $y \in S$. Otherwise it outputs 1.

Note that $S \in \mathsf{NP}$, and hence if $\mathsf{P} = \mathsf{NP}$, then $S \in \mathsf{P}$ also.

Clearly $\Pr_k[A(E(k, x_0)) = 0] = 1$, by definition of S (A always guesses the right string if $x = x_0$). Now let us show that there exists some $x_1 \in \{0,1\}^m$ such that $\Pr_k[A(E(k, x_1)) = 0] \leq 1/2$ (i.e. there exists some x_1 such that A guesses correctly that its x_1 with probability $\geq 1/2$).

For this, imagine we have a bipartite graph. On the left side, we put all codes of x_0, i.e., the left nodes are all elements of S.

The right nodes are every message $x \in \{0,1\}^m$.

Put an edge between $s \in S$ and $x' \in \{0,1\}^m$ if there exists some k such that $E(k, x') = s$.

Since each code-key pair must reveal the message, there are at most $|S|2^n = 2^{2n}$ edges in the graph.

But then there must exist some right-node $x_1 \in \{0,1\}^m$ having at most $2^{2n-m} \leq 2^{n-1}$ edges.

For this node, $\Pr_k[E(k, x_1) \in S] \leq \frac{1}{2}$, and hence $A(E(k, x_1)) = 1$ with probability $\geq 1/2$.

QED

How to get cryptography

Alright, so we need to assume that P != NP to get cryptography. Is this assumption enough? That is the most important open problem in cryptography!

But we can take another, stronger assumption, which is quite reasonable.

Definition. A function $\varepsilon : \mathbb{N} \to [0,1]$ is callled *negligible* if $\varepsilon(n) = n^{-\omega(1)}$, i.e., if it is less than every inverse-polynomial function. (less than $1/n$, less than $1/n^2$, less than $1/n^3$ etc for sufficiently large n)

Definition. A polynomial-time computable function $f : \{0,1\}^* \to \{0,1\}^*$ is called a *one-way function* if for every probabilistic polynomial-time algorithm A, the function

$$\varepsilon(n) = \Pr_{x \in \{0,1\}^n}[f(A(f(x))) = f(x)] \text{ is negligible.}$$

I.e., no ppt algorithm can successfully invert $f(x)$ for a random x.

Do you understand? The task of function inversion is, when given $f(x)$, to produce some x' such that $f(x') = f(x)$ (if f is one-to-one, it must follow that $x' = x$, but otherwise x' is just some preimage of $f(x)$).

One of the most important conjectures in all cryptography and all of complexity:

There exists a one-way function.

This conjecture is important for two reasons. First, it is plausible: it is really not obvious how one can invert a function. Second, it is *equivalent* to the existence of "computationally secure" cryptographic schemes. (we will define this notion later)

Examples of functions believed to be one-way

- Multiplication. The function $f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^{2n}$ given by $f(a, b) = a \cdot b$ is believed to be one-way. Inverting this function is equivalent to the task of prime factorization. The best algorithm known for factoring an n-digit integer runs in time $2^{O(n^{1/3} \log n)}$.
- Other examples using discrete log, lattices, etc, exist.
- See Section 9.2.2 of the book.

Ok, so now I make the following statement, of which I will only prove some parts:

The existence of one-way functions is equivalent to the existence of pseudorandom generators, and they are both equivalent to the existence of computationally-secure encryption schemes.

In this class we will now:

- define what a pseudorandom generator (PRG) is
- give one possible definition of what it means for a cryptographic scheme to be "computationally secure" (it turns out that all reasonable notions of computational security are equivalent, but that is not obvious, and is outside of the scope of this course)
- show that the existence of a PRG gives us the existence of a computationally secure cryptographic scheme
- Show that the existence of one-way *permutations* implies the existence of a PRG

We will not show the full equivalence between these different notions (it's complicated), namely, we will not show:

- That the existence of a one-way *function* implies the existence of a PRG. This is shown by taking any one-way function and converting it into a one-way *almost* permutation, and then using a proof similar to the one we will see, that works for *almost* permutations (whatever those are).
- The existence of a cryptographic scheme implies the existence of a one-way function. (I actually don't know how this is shown)

All of these results would be proven in a course on theoretical cryptography. The sum of these result together is that there exists a unified family of problems, which we could call "minicrypt", and the security of any problem in this class (against polynomial-time-bounded adversaries, such as Eve) is equivalent to the security of any other problem.

So: computational complexity is the ***right lens*** through which to study cryptography, even though cryptography existed for thousands of years.

As it turns out, the connection between the two areas is quite deep, and there is also a connection that goes the other way - the natural proofs barrier which we will discuss later.

PRGs

Definition (9.8 in book). Let $G : \{0,1\}^* \to \{0,1\}^*$ and $l(n) > n$ be such that $|G(x)| = l(|x|)$ for all $x \in \{0,1\}^*$.

We say that G is a *secure pseudorandom generator of stretch* $l(n)$ if for every probabilistic polynomial-time algorithm A,

$$|\Pr[A(G(U_n)) = 1] - \Pr[A(U_{l(n)}) = 1]| = n^{-\omega(1)} \text{ (i.e. is negligible)}$$

Let's process this definition.

It says that there exists no efficient algorithm for the following task:

Distinguishing task. I will give you either (a) the output $G(U_n)$ of the PRG on a uniformly-random input, or (b) a completely uniform string of the same length $l(n)$. Your task is to distinguish the two situations. I consider that you succeeded at this task if the probability of you accepting a pseudorandom string $G(U_n)$ is sufficiently different (non-negligible) than the probability of you accepting a truly random string $U_{l(n)}$.

you understand? you understand why this is called a "pseudorandom generator"?

The number of possible outputs $G(U_n)$ is at most 2^n whereas the total number of strings of that length is $2^{\ell(n)}$ which is at least twice as many (if $\ell(n) = n+1$) but could be a lot bigger.

So, naturally, *most strings of length* $l(n)$ are not in the image of G, i.e. are not pseudorandom. So what I'm asking you is to distinguish, when given some string, whether it is random or pseudorandom (and your strategy is allowed to depend on G, i.e. I've told you G beforehand).

Secure PRGs give us "computationally secure" encryption

As I mentioned, the notion of computationally secure encryption is fairly robust, but we will not show this here. We will settle by taking the following definition:

Definition (computational security). We will say that an encryption scheme (E, D) is computationally secure if for every probabilistic polynomial-time algorithm A,

$$\Pr_{k\in\{0,1\}^n, x\in\{0,1\}^m}[A(E(k,x)) = (i,b) \text{ such that } x_i = b] \leq \frac{1}{2} + n^{-\omega(1)}.$$

I.e., no efficient algorithm can guess any single bit of a random code.

Again, I emphasize, starting with an encryption scheme which is secure according to this definition, it is possible to obtain security against much more demanding definitions (like for every $x \neq x'$ and a random k, no efficient algorithm can distinguish $E(k,x)$ from $E(k,x')$)

PRGs imply computational security. Suppose there exists a secure PRG $G : \{0,1\}^* \to \{0,1\}^*$ with stretch $l(n) > n$. Then the following is a secure encryption scheme:

$$E(k,x) = G(k) \oplus x.$$

I.e., our key length is n, the input length of the PRG, and our message length is $l(n)$, the output length of the PRG. The encryption is then to take the "one-time pad" but where the pad is now the output of the PRG.

Proof. Suppose that some polynomial-time algorithm A allows Eve to break the encryption scheme (according to our def. of computational security). I.e., we have

$$\Pr_{k,x}[A(G(k) \oplus x) = (i,b) \text{ such that } x_i = b] \geq \frac{1}{2} + n^{-c} \text{ for some } c.$$

Let us then show that we can use the algorithm A to break the pseudorandom generator. I.e., we can use A to distinguish the output of G from a truly random string.

Our distinguisher D is as follows. We are given as input some string z, which is either a pseudorandom output $k' = G(k) \in \{0,1\}^{l(n)}$, or a truly random string $u \in \{0,1\}^{l(n)}$. (so $z \in \{k', u\}$ and we want to make a good guess of which is the case)

We then do as follows. Given z, we sample $x \in \{0,1\}^{l(n)}$ at random, compute $A(z \oplus x)$, and we output 1 if and only if $A(z \oplus x) = (i,b)$ with $x_i = b$. Now notice that:

- If $z = k' = G(k)$, then we output a correct pair (i,b) with probability significantly greater than $1/2$.
- But if $z = u$, then $z \oplus x$ is a completely uniform string independent of x! Hence it is independent of whether $x_i = 0$ or $x_i = 1$. So our guess (i,b) is such that $x_i = b$ with probability *exactly* $1/2$.

It then follows that

$$\Pr_k[D(G(k)) = 1] - \Pr[D(u) = 1] \geq \frac{1}{n^c}$$

which is non-negligible.

Goldreich-Levin

Ok, so we have shown that the existence of secure PRGs implies the existence of secure cryptographic schemes.

Now let's show that the existence of one-way *permutations* implies the existence of secure PRGs. (again, we will not do one-way functions, that proof is quite involved, but the basic idea is the same)

The theorem, in fact, is the following:

Theorem (Goldreich-Levin, '89). If $f : \{0,1\}^n \to \{0,1\}^n$ is a one-way function which is also a bijection, then

$$\Pr[A(f(x), r) = r \cdot x] \leq \frac{1}{2} + n^{-\omega(1)}.$$

where $r \cdot x = \sum_{i=1}^{n} r_i x_i \mod 2$ is the inner-product mod 2.

What does the theorem say?

1. It says that there exists no algorithm A which, when given as input $f(x)$ and r, can predict the value of $r \cdot x$.
2. It will follow that the map $G(x, r) = (f(x), r, r \cdot x)$, which accepts $2n$ bits and output $2n + 1$ is a pseudorandom generator.

Neither of the above two statements is obvious. But the second is easier to prove, so let's start there.

Unpredictability implies pseudorandomness

Definition. A function $G : \{0,1\}^n \to \{0,1\}^{l(n)}$ is called *unpredictable* if for every randomized polynomial-time algorithm B and every $i \in [l(n)]$,

$$\Pr_x[B(1^n, G(x)_1, \ldots, G(x)_{i-1}) = G(x)_i] \leq \frac{1}{2} + n^{-\omega(1)}.$$

I.e., G is unpredictable if there exists no efficient algorithm that can predict the i-th bit of the output $G(x)$ when given all the previous bits.

Theorem (Yao '82). If G is unpredictable, it is also a PRG.

Proof. Again the strategy is similar. We start by assuming that G is *not* a PRG, and we then try to prove that we can predict one of its bits.

So, let us be given an algorithm A that breaks G, i.e., A distinguishes a pseudorandom string from a truly random string:

$$|\Pr[A(G(U_n)) = 1] - \Pr[A(U_{l(n)}) = 1]| \geq \varepsilon$$

for some non-negligible $\varepsilon = \varepsilon(n)$

Let us now build an algorithm B for predicting one of the bits of $G(U_n)$.

This argument was invented by Yao, and is sometimes called the *hybrid method*, because we create hybrids of the two distributions $G(U_n)$ and $U_{l(n)}$.

For $i = 0, \ldots, n$, let D_i be the distribution $G(x)_1, \ldots, G(x)_{i-1}, z_i, \ldots, z_{\ell(n)}$, where each z bit is uniformly random. I.e., the first $i - 1$ bits are taken from the output of G and the remaining are uniformly random. Then A breaking the PRG means:

$$\begin{aligned}\varepsilon &\leq |\Pr[A(D_n)=1] - \Pr[A(D_0)=1]| \\ &= \left|\sum_{i=0}^{n-1} \Pr[A(D_{i+1})=1] - \Pr[A(D_i)=1]\right| \\ &\leq \sum_{i=0}^{n-1} |\Pr[A(D_{i+1})=1] - \Pr[A(D_i)=1]|\end{aligned}$$

So there must exist some i where

$$|\Pr[A(D_{i+1})=1] - \Pr[A(D_i)=1]| \geq \varepsilon/n$$

Let's look carefully at the two distributions D_{i+1} and D_i. They are:

$$D_{i+1} = G(x)_1 \dots G(x)_i, z_{i+1}, \dots, z_{\ell(n)} \qquad D_i = G(x)_1 \dots G(x)_{i-1}, z_i, \dots, z_{\ell(n)}$$

Then we know that A is ε/n more likely to accept if on position i I have the true value $G(x)_i$ versus if on position i I have a random bit z_i.

This might give us the following idea for a predicter B:

- On input $w_1 \dots w_{i-1}$, we generate random bits $z_i \dots z_{l(n)}$. We then compute $A(wz)$ and:
- If $A(wz)=1$ (comparatively more likely to happen if i-th bit is $G(x)_i$), then output z_i.
- If $A(wz)=0$ (comparatively less likely to happen if i-th bit is random), then output $1-z_i$.

Let us define:

$$p_{i+1} = \Pr[A(D_{i+1})=1] = \Pr[A(D_i)=1 \mid z_i = G(x)_i]$$

We also have that $\Pr[z_i = G(x)_i] = 1/2$, hence (Bayes rule):

$$\begin{aligned}p_i = \Pr[A(D_i)=1] &= \frac{1}{2}\Pr[A(D_i)=1 \mid z_i = G(x)_i] + \frac{1}{2}\Pr[A(D_i)=1 \mid z_i = 1-G(x)_i] \\ &= \frac{1}{2}p_{i+1} + \frac{1}{2}\Pr[A(D_i)=1 \mid z_i = 1-G(x)_i]\end{aligned}$$

which is to say

$$\Pr[A(D_i)=1 \mid z_i = 1-G(x)_i] = 2p_i - p_{i+1}.$$

Then:

$$\begin{aligned}\Pr[B(G(x)_1, \dots, G(x)_{i-1}) = G(x)_i)] &= \Pr[A(D_i)=1 \wedge z_i = G(x)_i] + \Pr[A(D_i)=0 \wedge z_i = 1-G(x)_i] \\ &= \frac{1}{2}(\Pr[A(D_i)=1 \mid z_i = G(x)_i] + (1 - \Pr[A(D_i)=1 \mid z_i = 1-G(x)_i])) \\ &= \frac{1}{2} + \frac{1}{2}(p_{i+1} - (2p_i - p_{i+1})) \\ &= \frac{1}{2} + p_{i+1} - p_i \geq \frac{1}{2} + \varepsilon/n\end{aligned}$$

Ok, so we predicted the next bit!

So we have proven: given a distinguisher that breaks the PRG G with breaking probability ε, we can obtain a predicter for one of the bits with predicting probability ε/n.

Conversely, if G is unpredictable, it is also a PRG.

Ok: Goldreich Levin is for next class!

So, next class we will do that, and then we will start discussing hardness vs randomness.

We will show how to take a function f which is hard to compute, and use it to make a Pseudorandom generator!

It will follow that, under very reasonable assumptions (that $E \nsubseteq SIZE(2^{o(n)})$), $\mathsf{P} = \mathsf{BPP}$!